Swift: Persistent Fault-Tolerant UAV-IoT Networks

Concept Map → Mathematical Formulation

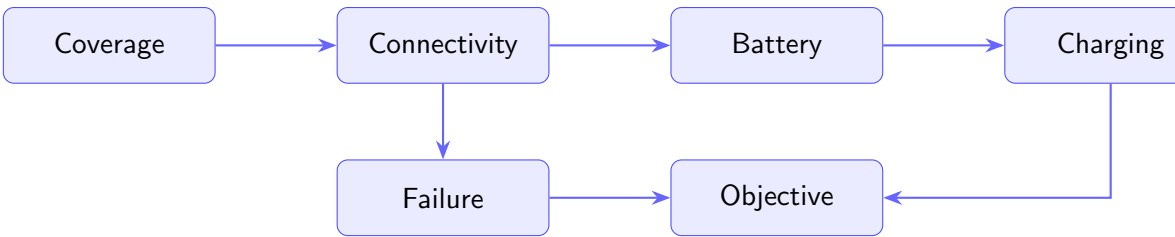
Team – Intelligent UAV Deployment

IIT Dharwad – ML in Wireless Networks

How to Read This Talk

Part A $\rightarrow$ Plain-language idea

Part B $\rightarrow$ Same idea in math

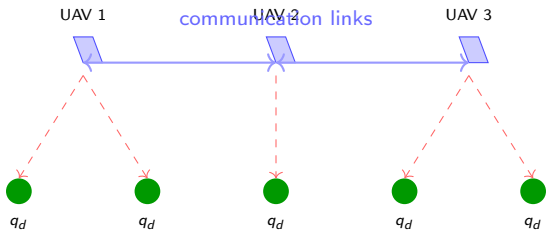


Each pair of slides: **idea** first, **equation** second.

The Problem (Idea)

We have N drones flying over M **ground sensors**.

Each drone has a battery that drains. Drones can **die**.
Sensors need **continuous coverage** — at least K drones above each.



Challenge: coordinate the drones with *no central controller* so coverage never drops.

The Problem (Math)

We model the swarm as a **Dec-POMDP**:

$$\mathcal{G} = \langle \underbrace{N}_{\text{agents}}, \underbrace{\mathcal{S}}_{\text{state}}, \underbrace{\{\mathcal{A}_i\}}_{\text{actions}}, \underbrace{\{\mathcal{O}_i\}}_{\text{observations}}, \underbrace{\mathcal{T}}_{\text{dynamics}}, \underbrace{R}_{\text{reward}}, \underbrace{\gamma}_{\text{discount}} \rangle$$

Objective: find a policy π that maximises expected cumulative reward:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t R_t\right]$$

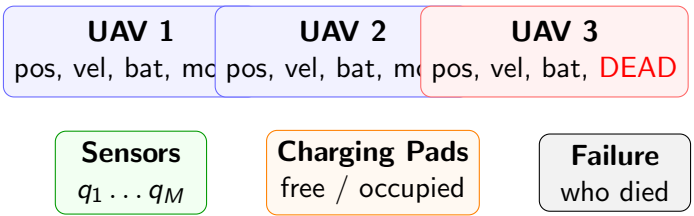
What each piece means

N	number of active UAV agents
$\mathcal{S}$	every possible system configuration
$\mathcal{A}_i$	what UAV i can do (move, hover, charge)
$\mathcal{O}_i$	what UAV i can see locally
$\mathcal{T}$	how the world transitions

What Does the World Look Like? (Idea)

Each second, the **entire system** has a snapshot:

1. **Where every drone is** and how much charge it has left.
2. **Where every ground sensor is** and whether it has enough drones above it.
3. **Which charging pads are free** and whether any drone just died.



This snapshot is the **state** s_t.

State Definition (Math)

$$s_t = \Big\{ \underbrace{(p_i,\ v_i,\ b_i,\ m_i)}_{\text{UAV } i \text{ info}},\quad \underbrace{(q_d)_{d=1}^{M}}_{\text{sensor positions}},\quad \underbrace{Q_t}_{\text{charging pads}},\quad \underbrace{F_t}_{\text{failures}} \Big\}$$

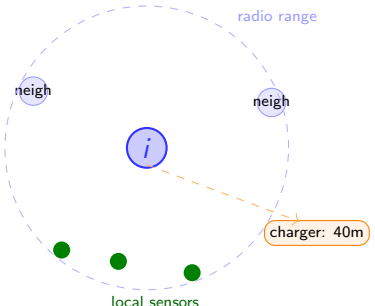
Symbol	Meaning	Range
p_i	position of UAV i	$(x, y, z) \in \mathbb{R}^3$
v_i	velocity of UAV i	$\|v_i\| \le v_{\max}$
b_i	battery level	$[0, 1]$ (normalised)
m_i	operating mode	{serve, transit, charge}
q_d	position of sensor d	fixed
Q_t	charging pad occupancy	$\{0, 1\}^S$
F_t	which UAVs failed	$\{0, 1\}^N$

Key point

What Can Each Drone See? (Idea)

A drone **cannot see everything**. It only knows:

- Its **own** position, velocity, battery.
- Which **neighbour drones** are nearby (within radio range).
- How many sensors are covered **in its local area**.
- Whether it is a **single point of failure** for connectivity.
- **How far** the nearest charging station is.



Local Observation (Math)

$$o_i = \Big[\underbrace{p_i,\ v_i,\ b_i}_{\text{own state}},\ \underbrace{\mathcal{N}_i}_{\text{neighbours}},\ \underbrace{C_i^{\text{local}}}_{\text{local coverage}},\ \underbrace{\text{risk}_i}_{\text{cut-vertex?}},\ \underbrace{t_i^{\text{empty}}}_{\text{time-to-empty}},\ \underbrace{E_i^{\text{station}}}_{\text{station energy}}\Big]$$

Symbol	Meaning
$\mathcal{N}_i$	indices of UAVs within R_{comm} of UAV i
C_i^{local}	how many sensors near UAV i are covered
risk_i	$= 1$ if removing UAV i disconnects the swarm
t_i^{empty}	seconds until battery hits the return threshold
E_i^{station}	energy cost to reach the nearest pad

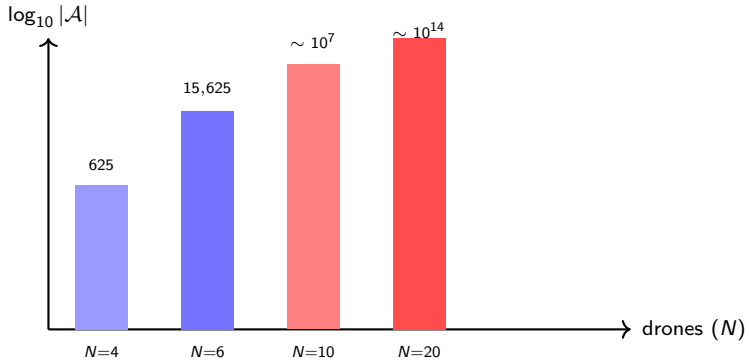
No central brain

At execution time every UAV runs its own policy $\pi_\theta(a_i \mid o_i)$ using only o_i — no global state needed.

Why Not One Central Controller? (Idea)

With 1 central controller choosing actions for all N drones:

Action space grows exponentially with the number of drones.



Solution: let each drone decide *for itself*, using only what it can see.

Decentralised Policy (Math)

Centralised: $|\mathcal{A}| = A^N$ where $A = |\mathcal{A}_i|$

Factorised (ours): each UAV picks its own action:

$$a_i \sim \pi_\theta(a_i \mid o_i) \qquad \Longrightarrow \qquad |\mathcal{A}|_{\text{eff}} = A \text{ (per agent)}$$

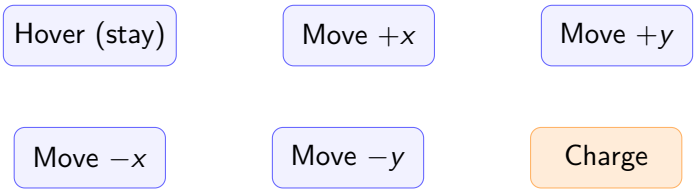
N	4	6	10	20
Centralised	5^4=625	5^6=15,625	$\sim 10^7$	$\sim 10^{14}$
Decentralised	5	5	5	5

Trade-off

Decentralised is tractable, but individual drones have *partial information*. The observation o_i must carry enough context for good coordination.

What Can a Drone Do? (Idea)

Each drone picks **one action per time step**:



Hover: stay in place, keep covering sensors.
Move: fly one step in a cardinal direction.
Charge: if at a pad, recharge battery.

Action Space (Math)

$$\mathcal{A}_i = \{\text{Hover},\ +x,\ -x,\ +y,\ -y,\ \text{Charge}\}, \quad |\mathcal{A}_i| = 6$$

Movement constraint:

$$p_i(t+1) = p_i(t) + v_i(t) \cdot \Delta t, \qquad \|v_i(t)\| \leq v_{\max}, \quad p_i \in \Omega$$

where Ω is the airspace boundary.

Charge action is only valid when:

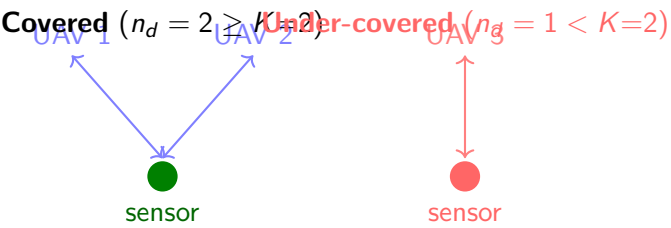
$$\|p_i - p_s^{\text{pad}}\| \leq d_{\text{charge}}, \quad Q_t[s] = 1 \text{ (pad is free)}$$

Otherwise the action defaults to *Hover*.

K-Coverage (Idea)

Rule

Every ground sensor must have **at least** K **drones** above it at all times.



We want **zero under-covered sensors** for as much of the mission as possible.

K-Coverage (Math)

Binary coverage — is UAV i covering sensor d?

$$x_{id}(t) = \mathbf{1}\big[\, \|p_i(t) - q_d\| \le R_{\text{cov}} \,\big]$$

Coverage count for sensor d:

$$n_d(t) = \sum_{i=1}^{N} x_{id}(t)$$

Coverage score (fraction of sensors adequately covered):

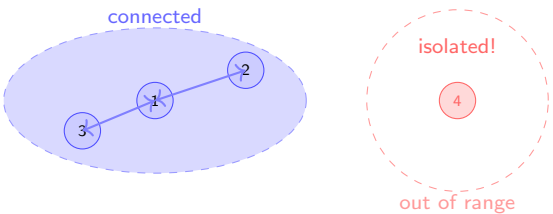
$$C(t) = \frac{1}{M} \sum_{d=1}^{M} \min\left(\frac{n_d(t)}{K},\ 1 \right)$$

Under-coverage rate (what we want to minimise):

$$U(t) = \frac{1}{M} \sum_{d=1}^{M} \mathbf{1}\big[\, n_d(t) < K \,\big]$$

Connectivity (Idea)

Drones must **talk to each other** to coordinate.
Two drones can communicate if they are within radio range R_{comm}.



If removing one drone **splits** the group, that drone is a **cut-vertex** — high risk.

Connectivity (Math)

Communication graph:

$$G_t = (V, E_t), \quad E_t = \{(i,j) : \|p_i - p_j\| \le R_{\text{comm}}\}$$

Connectivity risk — is UAV i a cut-vertex?

$$\text{risk}_i(t) = \mathbf{1}[\, G_t^{-i} \text{ is disconnected}\,]$$

where G_t^{-i} is the graph with UAV i and its edges removed.

Why this matters

A drone that is the *only link* between two parts of the swarm should **not** leave for charging without a replacement taking its place.

Desired global property:

$$G_t \text{ is connected} \quad \forall\, t$$

Battery Model (Idea)

Every drone burns energy doing three things:

1. **Hovering** — staying airborne.
2. **Moving** — flying to a new position.
3. **Communicating** — sending data to neighbours.



When battery hits the **return threshold**, the drone *must* go charge.

Battery Model (Math)

$$b_i(t+\Delta t) = b_i(t) + \underbrace{\frac{-P_{\text{hover}} - P_{\text{move}} - P_{\text{comm}}}{B_i^{\max}}\,\Delta t}_{\text{drain}} + \underbrace{\Delta b_i^{\text{charge}}}_{\text{recharge (at pad)}}$$

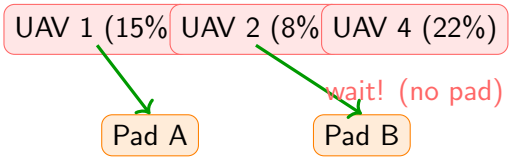
Return-to-charge condition:

$$b_i(t) \cdot B_i^{\max} \;\le\; E_i^{\text{req}} = E_i^{\text{station}} + E_i^{\text{return}} + E^{\text{reserve}}$$

Symbol	Meaning
$b_i(t)$	normalised battery $[0, 1]$
P_{hover}	power consumed hovering (W)
P_{move}	power consumed moving (W)
P_{comm}	power consumed communicating (W)
$B_i^{\max}$	total battery capacity (J)
E_i^{station}	energy to reach nearest pad

Charging Coordination (Idea)

Multiple drones may need to charge simultaneously, but pads are limited.



We solve a **min-cost assignment** every time a drone requests a pad.

Charging Coordination (Math)

Decision variable: $y_{is} \in \{0, 1\}$ — assign UAV i to pad s?

Cost:

$$c_{is} = \alpha\, d_{is} + \beta\, q_s + \mu\, u_i$$

where d_{is} = distance, q_s = queue length at pad, u_i = urgency.

Optimisation:

$$\min_{y} \sum_{i} \sum_{s} c_{is}\, y_{is}$$

Constraints:

$$\sum_{s} y_{is} \leq 1 \;\; \forall\, i \qquad \text{(each UAV} \leq \text{1 pad)}, \qquad \sum_{i} y_{is} \leq 1 \;\; \forall\, s \qquad \text{(each pad} \leq \text{1 UAV)}$$

Symbol	Meaning
α	weight for distance to station

The Goal (Idea)

We are balancing **6 competing objectives**:

Minimise under-coverage

Minimise energy waste

Minimise dead time

Minimise unnecessary moves

Minimise recovery time

Protect cut-vertices

Each objective gets a **weight** λ to control importance.

Multi-Objective Formulation (Math)

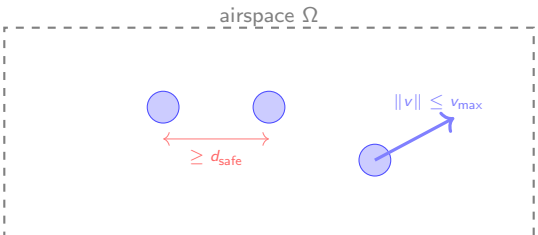
$$\min_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1}\Big(\underbrace{\lambda_O\, U(t)}_{\text{under-coverage}} + \underbrace{\lambda_E\, E(t)}_{\text{energy waste}} + \underbrace{\lambda_D\, D(t)}_{\text{dead time}} + \underbrace{\lambda_W\, W(t)}_{\text{wasted moves}} + \underbrace{\lambda_R\, R(t)}_{\text{recovery}} + \underbrace{\lambda_C\, \mathrm{Crit}(t)}_{\text{cut-vertex risk}}\Big)\right]$$

Term	Definition	What it penalises
$U(t)$	$\frac{1}{M}\sum_d \mathbf{1}[n_d < K]$	sensors below K-coverage
$E(t)$	fraction of flights with low battery	excessive energy drain
$D(t)$	fraction of time in degraded state	time after a failure
$W(t)$	moves that don't improve coverage	wasteful repositioning
$R(t)$	$\min(t_{\text{recovered}}, t_{\text{cap}})$	time to restore coverage
$\mathrm{Crit}(t)$	fraction of cut-vertices active	connectivity fragility

Safety Constraints (Idea)

Drones must follow hard rules at *every* time step:

1. **Speed limit** — don't fly too fast.
2. **Stay inside** the designated airspace.
3. **Don't collide** — maintain safe distance from other drones.



Safety Constraints (Math)

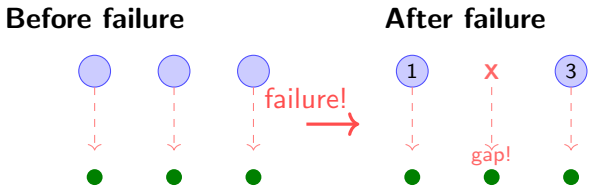
At every time step t, for all UAVs i:

1. **Speed:** $\|v_i(t)\| \leq v_{\max}$
2. **Workspace:** $p_i(t) \in \Omega \subset \mathbb{R}^3$
3. **Collision avoidance:** $\|p_i(t) - p_j(t)\| \geq d_{\text{safe}} \quad \forall j \neq i$

Hard vs soft

These are **hard constraints** — violated states are either prevented (action masking) or heavily penalised in the reward.

What Happens When a Drone Dies? (Idea)



We measure **how quickly** the remaining drones close the gap.

Failure & Recovery (Math)

Recovery time: how long until coverage is restored after a failure at t_f:

$$\tau_f = \inf\{\tau \geq 0 : n_d(t_f + \tau) \geq K \ \ \forall\, d \in \mathcal{M}\}$$

Requirement:

$$\tau_f \leq \tau_{\max}$$

Reserve UAV strategy

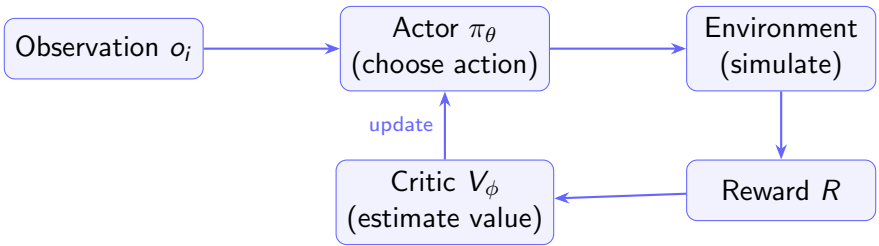
Keep 1 drone in reserve with $b_i > b_{\text{threshold}}$.
When a drone fails, the reserve immediately repositions to fill the gap, reducing τ_f.

Reserve activation condition:

reserve takes off when F_t changes (new failure detected)

How the AI Learns (Idea)

We use **MAPPO** — Multi-Agent Proximal Policy Optimisation.



Centralised training, decentralised execution: during training the critic sees all of s_t; at deployment each drone acts on o_i alone.

MAPPO (Math)

Actor (per UAV): $a_i \sim \pi_\theta(a_i \mid o_i)$

Critic (centralised, training only): $V_\phi(s_t)$

Actor loss (PPO clipped):

$$\mathcal{L}^{\text{actor}}(\theta) = -\mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t\Big)\Big]$$

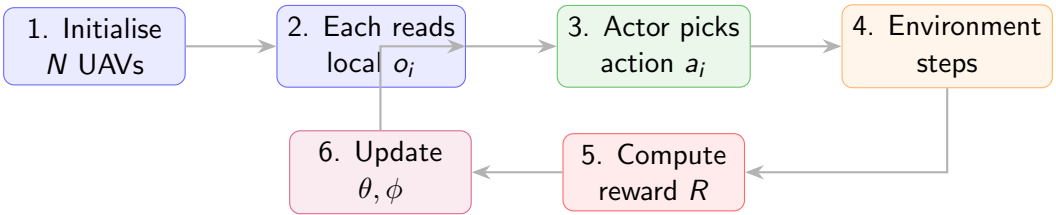
where $r_t(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{\text{old}}}(a_t|o_t)}$ and $\hat{A}_t = R_t - V_\phi(s_t)$ (GAE advantage).

Critic loss:

$$\mathcal{L}^{\text{critic}}(\phi) = \mathbb{E}_t\Big[\big(R_t - V_\phi(s_t)\big)^2\Big]$$

Hyperparameter	**Value**
Number of UAVs N	5
IoT devices M	20
Discount γ	0.99

Putting It All Together (Idea)



Implementation Summary (Math + Values)

Component	Math	Value
Airspace	Ω	$100 \times 100 \times 30$ m^3
UAVs	N	5 (+ 1 reserve)
Sensors	M	20
Coverage radius	R_{cov}	15 m
Comm radius	R_{comm}	40 m
Min K-coverage	K	2
Sim step	Δt	1 s
Max velocity	$v_{\max}$	5 m/s
Safe distance	d_{safe}	5 m
Battery capacity	$B^{\max}$	30 000 J
Hover power	P_{hover}	50 W
Move power	P_{move}	75 W
Comm power	P_{comm}	10 W
Charge rate	Δb^{charge}	0.01/step

Questions?